Project Discussion Summary and Work Log

Bi-LSTM Kubernetes autoscaling reproduction, NASA trace validation, and the cold-start-aware optimization. This report distinguishes user-provided history from code changes and measured evidence.

Executive summary

The repository implements the core offline predictor comparison and Algorithm 1 autoscaler simulation from Dang-Quang and Yoo (2021). The attached NASA logs were verified and the full NASA baseline was rerun on the corrected continuous minute series. A readiness-aware controller path was added and unit-tested. A live optimization win has **not** been measured: the current readiness p95 is shorter than one control interval, and the existing KIND API is unavailable.

Materials supplied in the conversation

- A Kubernetes/Prometheus workflow log describing a recommendationservice CrashLoopBackOff, its startupProbe fix, kube-prometheus-stack setup, Prometheus query checks, and CPU metric export.

- The 2021 Applied Sciences paper on Bi-LSTM-based proactive Kubernetes autoscaling, including its workload forecasting, cooling-down period, resource removal strategy, and evaluation tables.

- The NASA July/August 1995 access-log archives used for the paper's forecasting comparison.

Conversation and work timeline

Stage	Discussion and action	Outcome
Project identification	Compared the repository README and source with the attached workflow log and paper.	Project is a paper reproduction plus Kubernetes monitoring/autoscaling extensions.
Baseline assessment	Inspected predictors, Algorithm 1, saved tables, and the live controller.	Offline reproduction is substantial; the live hybrid controller is not the paper's Bi-LSTM controller.
Optimization design	Selected readiness-derived forecast lead time, recursive use of the one-step model, and scale-down protection while replicas are Pending.	A separate fixed-horizon comparison remains available through configuration.
Implementation	Added readiness p95 wiring, horizon selection/logging, tests, deployment helper, and experiment documentation.	24 planner assertions and recursive-forecast tests pass.
NASA validation	Validated raw files, repaired package imports, and made the loader restore missing zero-request minutes.	3,461,612 requests verified over 89,280 continuous minutes.
Presentation package	Reran the full NASA baseline in isolated Python 3.11 Docker and generated an evidence deck.	Fresh CSV, figures, PDF work log, and PowerPoint are saved under results/presentation_nasa_20260929.

Repository findings and implementation

What the repository supports

- The offline paper driver trains ARIMA, forward LSTM, and Bi-LSTM for one- and five-step forecasts, and computes prediction and elasticity metrics.

- The live KIND controller uses a Prophet-plus-residual-network hybrid. Its current experiment is distinct from the paper's Bi-LSTM/HPA comparison.

- Two Python import collisions were found and repaired: the data scripts resolved an empty src/data package instead of src/data.py, and the baseline model scripts resolved an empty src/models package instead of src/models.py.

- The shared loader now accepts both period/count and ds/y CSV layouts, groups duplicate minutes, sorts timestamps, and inserts zero counts for missing minutes.

Cold-start-aware controller changes

- Compute required lead steps from measured readiness p95 plus forecast overhead, divided by the trace-clock control interval.

- Recursively apply the packaged one-step forecaster up to a configured five-step cap; log required steps, used steps, and any uncovered lead.

- If readiness exceeds the forecast cap, log the shortfall and use recent observed demand as a conservative fallback.

- Hold scale-down while requested replicas are not all Ready. A scale-up cooldown still prevents repeated requests during startup.

- The readiness-aware mode is enabled in the controller manifest; setting READINESS_AWARE_SCALING=0 selects the fixed-horizon arm.

Key implementation files

File	Purpose
src/controller/config.py	Readiness measurements, horizon derivation, cap, and warning/report fields.
src/controller/forecaster.py	Recursive multi-step prediction using the existing one-step model.
src/controller/planner.py	Pending-replica scale-down guard.
src/controller/main.py	Horizon selection, safety fallback, and decision-log instrumentation.
k8s/controller.yaml	KIND runtime settings and measured readiness p95 value.
COLD_START_OPTIMIZATION.md	Policy, parameters, and paired-run instructions.

NASA dataset verification

The raw parser read 3,461,613 lines, skipped one malformed/truncated July record, and counted 3,461,612 requests. All observed timezone offsets were -0400. The regenerated full grid spans 1995-07-01 00:00 through 1995-08-31 23:59, has 89,280 minute bins, and contains 7,884 zero-request minutes. Both checked-in NASA CSVs match the raw counts at every timestamp they contain; the shared loader fills the 7,884 omitted bins for model input.

Measured results and limits

Refreshed full NASA run

One CPU run, seed 42, chronological 70/30 split, full training mode. RMSE is in requests per minute; relative difference is versus the cited paper Table 4 value.

Horizon	Model	Paper RMSE	Rerun RMSE	Delta
1-step	ARIMA	14.010	14.063	+0.4%
1-step	Bi-LSTM	13.551	13.788	+1.7%
5-step	ARIMA	15.414	15.481	+0.4%
5-step	Bi-LSTM	14.390	14.594	+1.4%

Interpretation: the refreshed NASA accuracy is close to the paper. It is not bit-identical because neural training is stochastic and hardware differs. The sparse CSV was correctly expanded to include zero-request minutes; old result tables are not assumed to represent this corrected input. The FIFA 98 tables were not rerun after the shared-loader change.

Readiness measurements and claim boundary

Quantity	Current value	Interpretation
Readiness p95	2.8662 s	Interpolated from 5 scale-out trials; preliminary tail estimate.
Control interval	6.0 s	One trace-minute at compression 10.
Selected lead	1 step	Readiness p95 is below one interval.
Optimization gain	Not measured	Under the saved startup condition, the optimized and fixed-horizon arms choose the same horizon.

The existing compression-sweep JSON is simulated, uses an unmeasured placeholder cold-start value, and explicitly notes that minute-resolution simulation cannot resolve sub-minute startup gains. It is not used as evidence of an optimization win.

KIND experiment status

The active context was kind-aiops-cluster. Its control-plane etcd and kube-apiserver repeatedly exited, with containerd reporting stale reserved pod-sandbox names. Restarting containerd did not restore the API. No cluster deletion or recreation was attempted. Therefore the live paired A/B was not run.

Next steps for a defensible optimization result

- Restore a healthy KIND cluster without discarding its persisted state, or provision a new, separately named test cluster.

- Collect at least 20 readiness measurements and include a controlled startup-under-load condition with p95 above 6 seconds.

- Run fixed-horizon and readiness-aware arms on identical trace segments, model, capacity calibration, replica bounds, and load.

- Report p95/p99 latency and SLO violations together with replica-seconds, CPU-seconds, pending-pod time, scaling actions, and confidence intervals.

- Keep the paper-faithful Bi-LSTM + CDT/RRS simulation separate from the live hybrid-controller experiment.

Validation performed

NASA preprocessing sanity gates passed; two dataset regression tests passed; 24 planner assertions passed; recursive horizon tests passed; experiment CLIs imported; Python compilation and manifest consistency checks passed. The seven-slide PowerPoint file was reopened and validated with python-pptx.

Generated artifacts

results/presentation_nasa_20260929/cold_start_aware_evidence.pptx
results/presentation_nasa_20260929/EVIDENCE.md
results/presentation_nasa_20260929/tables/table_nasa_accuracy.csv
results/presentation_nasa_20260929/figures/